

Codebook

Contents

1 basic	2		
1.1 build	2	5.5 chinese remainder theorem coprime	9
1.2 default	2	5.6 chinese remainder theorem not coprime	9
1.3 div	2	5.7 count prime	9
1.4 fast	2	5.8 det	10
1.5 pragma	2	5.9 extgcd	10
1.6 random	2	5.10 factorize	10
2 data structure	2	5.11 fraction	10
2.1 2d segment tree	2	5.12 gaussian elimination	10
2.2 bit	3	5.13 josephus	11
2.3 bitset	3	5.14 lucas	11
2.4 lichao	3	5.15 math block	11
2.5 pbds	3	5.16 miller rabin	11
2.6 range set	3	5.17 phi	11
2.7 sparse table	3	5.18 phi lemma	12
2.8 STL	4	5.19 primes er	12
2.9 treap	4	5.20 primes linear	12
2.10 undo dsu	4	5.21 two equations	12
2.11 unordered map	4	6 other	12
3 flow	4	6.1 aliens dp	12
3.1 dinic	4	6.2 all LCS	12
3.2 mcmf	5	6.3 dc dp	12
4 graph	5	6.4 digit dp	12
4.1 2SAT	5	6.5 geometry	13
4.2 bridge	5	6.6 line container	13
4.3 bridge bcc	6	6.7 mo	13
4.4 cut	6	6.8 number split	13
4.5 cut bcc	6	6.9 sos dp	13
4.6 matching dfs	6	6.10 subset	13
4.7 matching general	6	6.11 xor basis	14
4.8 matching general weight	7	7 polynomial	14
4.9 prim	8	7.1 FFT	14
4.10 scc	8	7.2 NTT	14
4.11 spfa	8	7.3 NTT prime	14
5 math	8	7.4 operation	14
5.1 big number	8	8 string	15
5.2 burnside lemma	9	8.1 hash	15
5.3 catalan	9	8.2 kmp	16
5.4 chinese remainder theorem coprime	9	8.3 manacher	16
		8.4 string trie	16
		8.5 xor trie	16
		8.6 z	16
		9 tree	17
		9.1 hld	17

1 basic

1.1 build

```

1 code tem.cpp
2 ...
3 for i in {A..M}; do cp tem.cpp \${i}.cpp; done;
4 code *.cpp
5 code run
6
7 寫入以下編譯與執行指令至 run 檔案中：
8 g++ -std=c++20 -DABS -Wall -Wextra -Wshadow $1.cpp -o $1 && ./
9   $1
10 chmod +x run

```

1.2 default

```

1 #include <bits/stdc++.h>
2 #define int long long
3 #define pii pair<int, int>
4 #define F first
5 #define S second
6 #define pb push_back
7 #define all(x) x.begin(), x.end()
8 #define sz(x) (int)x.size()
9 #define FOR(i, a, b) for(int i = a; i <= b; ++i)
10 #define IO ios::sync_with_stdio(0), cin.tie(0)
11 using namespace std;
12 void debug() { cerr << endl; }
13 template <typename T, typename ...U>
14 void debug(T i, U ...j) { cerr << i << ' ', debug(j...); }
15 #define test(...) debug("[ " #__VA_ARGS__ " ]:", __VA_ARGS__)
16
17 signed main() {
18     IO;
19
20 }

```

1.3 div

```

1 ll floor(ll a, ll b) {
2     return a / b - (a < 0 && a % b);
3 }
4 ll ceil(ll a, ll b) {
5     return a / b + (a > 0 && a % b);
6 }
7 a / b < x -> floor(a, b) + 1 <= x
8 a / b <= x -> ceil(a, b) <= x
9 x < a / b -> x <= ceil(a, b) - 1
10 x <= a / b -> x <= floor(a, b)

```

1.4 fast

```

1 #pragma GCC optimize("Ofast,inline,unroll-loops")
2 #pragma GCC target("bmi,bmi2,lzcnt,popcnt,avx2")
3 #define windows_system
4 #ifdef windows_system
5     /* For Windows */
6 inline int getchar_unlocked() { return getchar(); }
7 inline void putchar_unlocked(char _c) { putchar(_c); }
8 #endif
9 inline int in() {
10     int re = 0;
11     char c = getchar_unlocked();
12     while (c == ' ' || c == '\n') c = getchar_unlocked();
13     while (c >= '0' && c <= '9') re = (re << 3) + (re << 1) + c - '0', c = getchar_unlocked();
14     return re;
15 }
16 inline void out(int x) {
17     char str[20];
18     int pos = 0;
19     do {
20         str[pos++] = x % 10 + '0';
21         x /= 10;
22     } while (x);
23     for (int i = pos - 1; i >= 0; i--) putchar_unlocked(str[i]);
24     putchar_unlocked('\n');
25 }

```

1.5 pragma

```

1 #pragma GCC optimize("Ofast,no-stack-protector")
2 #pragma GCC optimize("no-math-errno,unroll-loops")
3 #pragma GCC target("sse,sse2,sse3,ssse3,sse4")
4 #pragma GCC target("popcnt,abm,mmx,avx,arch=skylake")

```

1.6 random

```

1 mt19937 seed(chrono::steady_clock::now().time_since_epoch().
2   count());
3 inline int rnd(int l, int r) {
4     return uniform_int_distribution<int>(l, r)(seed);
5 }
6 inline double drnd(double l, double r) {
7     return uniform_real_distribution<double>(l, r)(seed);
8 }

```

2 data structure

2.1 2d segment tree

```

1 struct Node2 {
2     int val; Node2 *lc, *rc;
3     Node2 () {
4         val = 0;
5         lc = rc = nullptr;
6     }
7 };
8 struct Node1 {
9     Node2 *c; Node1 *lc, *rc;
10    Node1 () {
11        c = nullptr;
12        lc = rc = nullptr;
13    }
14 };
15 int N;
16 int Val2(Node2 *n2) { return n2 ? n2->val : 0; }
17 void pull2(Node2 *n2) { n2->val = Val2(n2->lc) + Val2(n2->rc); }
18 void update2(Node2 *n2, int l, int r, int id, int v) {
19     if (l == r) {
20         n2->val += v; return;
21     }
22     int mid = (l + r) >> 1;
23     if (id <= mid) {
24         if (!n2->lc) n2->lc = new Node2();
25         update2(n2->lc, l, mid, id, v);
26     } else {
27         if (!n2->rc) n2->rc = new Node2();
28         update2(n2->rc, mid + 1, r, id, v);
29     }
30     pull2(n2);
31 }
32 void update1(Node1 *n1, int l, int r, int qx, int qy, int v) {
33     if (!n1->c) n1->c = new Node2();
34     update2(n1->c, l, N, qx, v);
35     if (l == r) return;
36     int mid = (l + r) >> 1;
37     if (qx <= mid) {
38         if (!n1->lc) n1->lc = new Node1();
39         update1(n1->lc, l, mid, qx, qy, v);
40     } else {
41         if (!n1->rc) n1->rc = new Node1();
42         update1(n1->rc, mid + 1, r, qx, qy, v);
43     }
44 }
45 int query2(Node2 *n2, int l, int r, int ql, int qr) {
46     if (!n2) return 0;
47     if (ql <= l && r <= qr) return n2->val;
48     int mid = (l + r) >> 1;
49     if (qr <= mid)
50         return query2(n2->lc, l, mid, ql, qr);
51     else if (mid + 1 <= ql)
52         return query2(n2->rc, mid + 1, r, ql, qr);
53     else
54         return query2(n2->lc, l, mid, ql, qr) + query2(n2->rc, mid + 1, r, ql, qr);
55 }
56 int query1(Node1 *n1, int l, int r, int qxl, int qxr, int qyl,
57   int qyr) {
58     if (!n1) return 0;
59     if (qxl <= l && r <= qxr) return query2(n1->c, l, N, qyl, qyr);
60     int mid = (l + r) >> 1;
61     if (qxr <= mid)
62         return query1(n1->lc, l, mid, qxl, qxr, qyl, qyr);
63     else if (mid + 1 <= qxl)

```

```

63     return query1(n1->rc, mid + 1, r, qxl, qxr, qyl, qyr);
64 else
65     return query1(n1->lc, l, mid, qxl, qxr, qyl, qyr) + query1(
66         n1->rc, mid + 1, r, qxl, qxr, qyl, qyr);

```

2.2 bit

```

1 struct BIT {
2     int N;
3     vector<int> info;
4     BIT(int n) : N(n), info(n + 1) {}
5     void update(int idx, int val) {
6         for (int i = idx; i <= N; i += i & -i)
7             info[i] += val;
8     }
9     int query(int idx) {
10        int res = 0;
11        for (int i = idx; i > 0; i -= i & -i)
12            res += info[i];
13        return res;
14    }
15    int kth(int k) { // kth small
16        int res = 0;
17        for (int i = 1 << __lg(N); i > 0; i >= 1) {
18            if (res + i <= N && info[res + i] < k) {
19                res += i;
20                k -= info[res];
21            }
22        }
23        return res + 1;
24    }
25 };

```

2.3 bitset

```

1 #pragma GCC target("popcnt")
2 // 15 = 1111
3 bitset<10> bit(15); // 宣告長度 = 10 的 bitset
4 cout << bit.count() << "\n"; // 回傳幾個 1
5 cout << bit.none() << "\n"; // 回傳是否全為 0
6 cout << bit.any() << "\n"; // 回傳是否有 1
7 cout << bit._Find_first() << "\n"; // 回傳第一個 bit = 1 的位
8 // 置
9 int x = 2;
10 cout << bit._Find_next(x) << "\n"; // 回傳 x 後 · 第一個 bit =
11 // 1 的位置

```

2.4 lichao

```

1 // MIN LeeChao Segment Tree
2 struct line {
3     int m, k;
4     line *lc, *rc;
5     line() { m = 0, k = INT_MAX, lc = nullptr, rc = nullptr; }
6     line(int _m, int _k) { m = _m, k = _k, lc = nullptr, rc =
7         nullptr; }
8     int calc(int x) {
9         return m * x + k;
10    }
11    void push(int l, int r) {
12        if (l == r) return;
13        if (!lc) lc = new line();
14        if (!rc) rc = new line();
15    }
16 };
17 struct LeeChao {
18     line* root;
19     LeeChao() : root(new line()) {}
20     void update(line* node, int l, int r, line add) {
21         if (l == r) {
22             if (add.calc(l) < node->calc(l)) {
23                 swap(add.m, node->m);
24                 swap(add.k, node->k);
25             }
26             return;
27         }
28         // 預設 seg[node] 的斜率 < add 的斜率
29         if (node->m > add.m) {
30             swap(add.m, node->m);
31             swap(add.k, node->k);
32         }
33         node->push(l, r);
34         int mid = (l + r) >> 1;
35         // 將小的 f(x) 留下
36         if (node->calc(mid) > add.calc(mid)) {

```

```

36         swap(add.m, node->m);
37         swap(add.k, node->k);
38         update(node->rc, mid + 1, r, add);
39     } else {
40         update(node->lc, l, mid, add);
41     }
42 }
43 int query(line* node, int l, int r, int x) {
44     if (node->k == INT_MAX) return INT_MAX;
45     if (l == r) return node->calc(x);
46     int mid = (l + r) >> 1;
47     if (x <= mid)
48         return min(node->calc(x), query(node->lc, l, mid, x));
49     else
50         return min(node->calc(x), query(node->rc, mid + 1, r, x));
51 }
52 void update(line l) { update(root, 1, N, l); }
53 int query(int x) { return query(root, 1, N, x); }
54 };

```

2.5 pbds

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 void solve() {
5     tree<int, null_type, greater<int>, rb_tree_tag,
6         tree_order_statistics_node_update> bst;
7     bst.insert(1);
8     int a = *bst.find_by_order(2); // 第 k + 1 大的數
9     int b = bst.order_of_key(2); // 有幾個數 > k
10    // -----
11    tree<int, null_type, less<int>, rb_tree_tag,
12        tree_order_statistics_node_update> bst;
13    bst.insert(1);
14    int a = *bst.find_by_order(2); // 第 k + 1 小的數
15    int b = bst.order_of_key(2); // 有幾個數 < k
16 }

```

2.6 range set

```

1 struct RangeSet { // [L, r)
2     set<pii> st;
3     void cut(int x) {
4         auto it = st.lower_bound({x + 1, -1});
5         if (it == st.begin()) return;
6         auto [l, r] = *prev(it);
7         if (l >= x || x >= r) return;
8         st.erase(prev(it));
9         st.insert({l, x});
10        st.insert({x, r});
11    }
12    vector<pii> split(int l, int r) {
13        // remove and return ranges in [L, r)
14        cut(l), cut(r);
15        vector<pii> res;
16        while (true) {
17            auto it = st.lower_bound({l, -1});
18            if (it == st.end() || r <= it->first) break;
19            res.push_back(*it), st.erase(it);
20        }
21        return res;
22    }
23    void insert(int l, int r) {
24        // add a range [L, r), [L, r) not in st
25        auto it = st.lower_bound({l, r});
26        if (it != st.begin() && prev(it)->second == l)
27            l = prev(it)->first, st.erase(prev(it));
28        if (it != st.end() && r == it->first)
29            r = it->second, st.erase(it);
30        st.insert({l, r});
31    }
32    bool count(int x) {
33        auto it = st.lower_bound({x + 1, -1});
34        return it != st.begin() && prev(it)->first <= x && x < prev(
35            it)->second;
36    };

```

2.7 sparse table

```

1 int a[N];
2 struct Sparse_table {
3     int n;
4     vector<vector<int>> st;

```

```

5 Sparse_table(int _n) : n(_n), st(_n + 1, vector<int>(__lg(_n)
6 + 1)) {
7     for (int i = 1; i <= n; i++) st[i][0] = a[i];
8     for (int lg = 1; lg <= __lg(n); lg++) {
9         int len = 1 << (lg - 1);
10        for (int i = 1; i + len <= n; i++)
11            st[i][lg] = max(st[i][lg - 1], st[i + len][lg - 1]);
12    }
13    int query(int l, int r) {
14        int lg = __lg(r - l + 1);
15        int len = 1 << lg;
16        return max(st[l][lg], st[r - len + 1][lg]);
17    }
18 };

```

2.8 STL

```

1 struct Data {
2     int x, y;
3 };
4 // priority_queue
5 struct cmp {
6     bool operator()(Data a, Data b) {
7         return a.x < b.x;
8     }
9 };
10 priority_queue<Data, vector<Data>, cmp> pq;
11 // set
12 struct cmp {
13     bool operator()(Data p, Data q) {
14         return p.x < q.x;
15     }
16 };
17 set<Data, cmp> st;

```

2.9 treap

```

1 struct node {
2     int data, sz;
3     node *l, *r;
4     node(int k) : data(k), sz(1), l(0), r(0) {}
5     void up() {
6         sz = 1;
7         if (l) sz += l->sz;
8         if (r) sz += r->sz;
9     }
10    void down() {}
11 };
12 int sz(node *a) { return a ? a->sz : 0; }
13 node *merge(node *a, node *b) {
14     if (!a || !b) return a ? a : b;
15     if (rand() % (sz(a) + sz(b)) < sz(a))
16         return a->down(), a->r = merge(a->r, b), a->up(), a;
17     return b->down(), b->l = merge(a, b->l), b->up(), b;
18 }
19 void split1(node *o, node *&a, node *&b, int k) {
20     if (!o) return a = b = 0, void();
21     o->down();
22     if (o->data <= k)
23         a = o, split1(o->r, a->r, b, k), a->up();
24     else
25         b = o, split1(o->l, a, b->l, k), b->up();
26 }
27 void split2(node *o, node *&a, node *&b, int k) {
28     if (sz(o) <= k) return a = o, b = 0, void();
29     o->down();
30     if (sz(o->l) + 1 <= k)
31         a = o, split2(o->r, a->r, b, k - sz(o->l) - 1);
32     else
33         b = o, split2(o->l, a, b->l, k);
34     o->up();
35 }
36 node *kth(node *o, int k) {
37     if (k <= sz(o->l)) return kth(o->l, k);
38     if (k == sz(o->l) + 1) return o;
39     return kth(o->r, k - sz(o->l) - 1);
40 }
41 int Rank(node *o, int key) {
42     if (!o) return 0;
43     if (o->data < key)
44         return sz(o->l) + 1 + Rank(o->r, key);
45     else
46         return Rank(o->l, key);
47 }
48 bool erase(node *&o, int k) {
49     if (!o) return 0;
50     if (o->data == k) {
51         node *t = o;
52         o->down(), o = merge(o->l, o->r);
53         delete t;

```

```

54     return 1;
55 }
56 node *t = k < o->data ? o->l : o->r;
57 return erase(t, k) ? o->up(), 1 : 0;
58 }
59 void insert(node *&o, int k) {
60     node *a, *b;
61     split(o, a, b, k),
62     o = merge(a, merge(new node(k), b));
63 }
64 void interval(node *&o, int l, int r) {
65     node *a, *b, *c;
66     split2(o, a, b, l - 1), split2(b, b, c, r);
67     // operate
68     o = merge(a, merge(b, c));
69 }

```

2.10 undo dsu

```

1 struct DSU {
2     vector<int> parent, size, tag;
3     vector<tuple<int, int, int, int, int>> history;
4     DSU(int n) {
5         parent.resize(n + 1);
6         size.resize(n + 1, 1);
7         tag.resize(n + 1, 0);
8         iota(all(parent), 0);
9     }
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return find(parent[x]);
13    }
14    bool merge(int x, int y) {
15        x = find(x), y = find(y);
16        if (x == y) return false;
17        if (size[x] < size[y]) swap(x, y);
18        history.pb({x, y, parent[y], tag[x], size[x]});
19        parent[y] = x;
20        size[x] += size[y];
21        return true;
22    }
23    void add_tag(int head, int val) {
24        tag[head] += val;
25    }
26    void undo() {
27        auto [px, py, parent_py, base, sz_x] = history.back();
28        parent[py] = parent_py;
29        size[px] = sz_x;
30        int delta = tag[px] - base;
31        tag[py] += delta;
32        history.pop_back();
33    }
34 };

```

2.11 unordered map

```

1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t a) const {
9         static const uint64_t FIXED_RANDOM = chrono::steady_clock::
10            now().time_since_epoch().count();
11         return splitmix64(i + FIXED_RANDOM);
12     }
13 };
14 unordered_map<int, int, custom_hash> m1;

```

3 flow

3.1 dinic

```

1 struct Flow {
2     struct Edge {
3         // rc : residual capacity
4         int u, rc, rev;
5     };
6     vector<vector<Edge>> G;
7     vector<int> dis, it;
8     Flow(int n) : G(n), dis(n), it(n) {}
9     void add(int v, int u, int c) {
10         G[v].push_back({u, c, sz(G[u])});
11         G[u].push_back({v, 0, sz(G[v]) - 1});
12     }

```

```

13 int dfs(int v, int t, int f) {
14     if (v == t || f == 0) return f;
15     for (int &i = it[v]; i < sz(G[v]); i++) {
16         auto &[u, rc, rev] = G[v][i];
17         if (dis[u] != dis[v] + 1) continue;
18         int res = dfs(u, t, min(f, rc));
19         if (res <= 0) continue;
20         rc -= res;
21         G[u][rev].rc += res;
22         return res;
23     }
24     return 0;
25 }
26 int flow(int s, int t) {
27     int ans = 0;
28     for (int l = 30; l >= 0; l--) while (true) {
29         fill(all(dis), INT_MAX);
30         queue<int> Q;
31         Q.push(s);
32         dis[s] = 0;
33         while (!Q.empty()) {
34             int v = Q.front(); Q.pop();
35             for (auto [u, rc, rev] : G[v]) {
36                 if ((rc >> 1) <= 0 || dis[u] < INT_MAX) continue;
37                 dis[u] = dis[v] + 1;
38                 Q.push(u);
39             }
40         }
41         if (dis[t] == INT_MAX) break;
42
43         fill(all(it), 0);
44         while (true) {
45             int res = dfs(s, t, INT_MAX);
46             if (res <= 0) break;
47             ans += res;
48         }
49     }
50     return ans;
51 }
52 };

```

3.2 mcmf

```

1 struct Flow {
2     struct Edge {
3         int u, rc, k, rv;
4     };
5     vector<vector<Edge>> G;
6     vector<int> par, par_eid;
7     Flow(int n) : G(n + 1), par(n + 1), par_eid(n + 1) {}
8     void add(int v, int u, int c, int k) {
9         G[v].push_back({u, c, k, (int)G[u].size()});
10        G[u].push_back({v, 0, -k, (int)G[v].size() - 1});
11    }
12    int spfa(int s, int t) {
13        fill(par.begin(), par.end(), -1);
14        vector<int> dis(par.size(), LONG_LONG_MAX);
15        vector<bool> in_q(par.size(), false);
16        queue<int> Q;
17        dis[s] = 0; in_q[s] = true;
18        Q.push(s);
19        while (!Q.empty()) {
20            int v = Q.front(); Q.pop();
21            in_q[v] = false;
22            for (int i = 0; i < (int)G[v].size(); i++) {
23                auto [u, rc, k, rv] = G[v][i];
24                if (rc > 0 && dis[v] + k < dis[u]) {
25                    dis[u] = dis[v] + k;
26                    par[u] = v;
27                    par_eid[u] = i;
28                    if (!in_q[u]) Q.push(u);
29                    in_q[u] = true;
30                }
31            }
32        }
33        return dis[t];
34    }
35    pair<int, int> flow(int s, int t) {
36        int fl = 0, cost = 0, d;
37        while ((d = spfa(s, t)) < LONG_LONG_MAX) {
38            int cur = LONG_LONG_MAX;
39            for (int v = t; v != s; v = par[v])
40                cur = min(cur, G[par[v]][par_eid[v]].rc);
41            fl += cur;
42            cost += d * cur;
43            for (int v = t; v != s; v = par[v]) {
44                G[par[v]][par_eid[v]].rc -= cur;
45                G[v][G[par[v]][par_eid[v]].rv].rc += cur;
46            }
47        }
48        return {fl, cost};
49    }
50 };

```

4 graph

4.1 2SAT

```

1 struct TWO_SAT {
2     int n, N;
3     vector<vector<int>> G, rev_G;
4     deque<bool> used;
5     vector<int> order, comp;
6     deque<bool> assignment;
7     void init(int _n) {
8         n = _n;
9         N = _n * 2;
10        G.resize(N + 5);
11        rev_G.resize(N + 5);
12    }
13    void dfs1(int v) {
14        used[v] = true;
15        for (int u : G[v]) {
16            if (!used[u])
17                dfs1(u);
18        }
19        order.push_back(v);
20    }
21    void dfs2(int v, int c1) {
22        comp[v] = c1;
23        for (int u : rev_G[v]) {
24            if (comp[u] == -1)
25                dfs2(u, c1);
26        }
27    }
28    bool solve() {
29        order.clear();
30        used.assign(N, false);
31        for (int i = 0; i < N; ++i) {
32            if (!used[i])
33                dfs1(i);
34        }
35        comp.assign(N, -1);
36        for (int i = 0, j = 0; i < N; ++i) {
37            int v = order[N - i - 1];
38            if (comp[v] == -1)
39                dfs2(v, j++);
40        }
41        assignment.assign(n, false);
42        for (int i = 0; i < N; i += 2) {
43            if (comp[i] == comp[i + 1])
44                return false;
45            assignment[i / 2] = (comp[i] > comp[i + 1]);
46        }
47        return true;
48    }
49    void add_disjunction(int a, bool na, int b, bool nb) { // A
50        // or B
51        // na means whether a is negative or not
52        // nb means whether b is negative or not
53        a = 2 * a ^ na;
54        b = 2 * b ^ nb;
55        int neg_a = a ^ 1;
56        int neg_b = b ^ 1;
57        G[neg_a].push_back(b);
58        G[neg_b].push_back(a);
59        rev_G[b].push_back(neg_a);
60        rev_G[a].push_back(neg_b);
61        return;
62    }
63    void get_result(vector<int>& res) {
64        res.clear();
65        for (int i = 0; i < n; i++)
66            res.push_back(assignment[i]);
67    }
68 };

```

4.2 bridge

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N];
5 vector<pair<int, int>> bridge;
6 void dfs(int v, int p) {
7     vis[v] = true;
8     low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
9     for (int u : G[v]) {
10        if (u == p) continue;
11        if (!vis[u]) {
12            // (v, u) 是樹邊
13            dfs(u, v);
14            low[v] = min(low[v], low[u]);
15        } else {

```

```

16     // (v, u) 是回邊
17     low[v] = min(low[v], depth[u]);
18 }
19 }
20 // v 在不依靠父邊的情況下永遠沒辦法走到它的祖先
21 // (root, -1) 不算
22 if (low[v] == depth[v] && p != -1)
23     bridge.push_back({v, p});
24 }

```

4.3 bridge bcc

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N];
5 vector<vector<int>> bcc;
6 stack<int> stk;
7 void dfs(int v, int p) {
8     stk.push(v);
9     vis[v] = true;
10    low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
11    for (int u : G[v]) {
12        if (u == p) continue;
13        if (!vis[u]) {
14            // (v, u) 是樹邊
15            dfs(u, v);
16            low[v] = min(low[v], low[u]);
17        } else {
18            // (v, u) 是回邊
19            low[v] = min(low[v], depth[u]);
20        }
21    }
22    // v 在不依靠父邊的情況下永遠沒辦法走到它的祖先
23    if (low[v] == depth[v]) {
24        bcc.emplace_back();
25        while (stk.top() != v) {
26            bcc.back().push_back(stk.top());
27            stk.pop();
28        }
29        bcc.back().push_back(stk.top());
30        stk.pop();
31    }
32 }

```

4.4 cut

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N], cut[N];
5 void dfs(int v, int p) {
6     vis[v] = true;
7     int child = 0;
8     low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
9     for (int u : G[v]) {
10        if (u == p) continue;
11        if (!vis[u]) {
12            // (v, u) 是樹邊
13            dfs(u, v);
14            child++;
15            low[v] = min(low[v], low[u]);
16            // u 無法在不經過父邊的情況走到 v 的祖先
17            if (low[u] >= depth[v] && p != -1)
18                cut[v] = true;
19        } else {
20            // (v, u) 是回邊
21            low[v] = min(low[v], depth[u]);
22        }
23    }
24    // 根節點有超過 2 個子節點
25    if (p == -1 && child >= 2)
26        cut[v] = true;
27 }

```

4.5 cut bcc

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N];
5 vector<vector<int>> bcc;
6 stack<int> stk;
7 void dfs(int v, int p) {
8     stk.push(v);
9     vis[v] = true;

```

```

10    low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
11    for (int u : G[v]) {
12        if (u == p) continue;
13        if (!vis[u]) {
14            // (v, u) 是樹邊
15            dfs(u, v);
16            low[v] = min(low[v], low[u]);
17            // u 無法在不經過父邊的情況走到 v 的祖先
18            if (low[u] >= depth[v]) {
19                bcc.emplace_back();
20                while (stk.top() != u) {
21                    bcc.back().push_back(stk.top());
22                    stk.pop();
23                }
24                bcc.back().push_back(stk.top());
25                stk.pop();
26                bcc.back().push_back(v);
27            }
28        } else {
29            // (v, u) 是回邊
30            low[v] = min(low[v], depth[u]);
31        }
32    }
33 }

```

4.6 matching dfs

```

1 const int N = 2005;
2 vector<int> G[N];
3 struct Matching {
4     int n;
5     vector<bool> vis; vector<int> match;
6     Matching(int _n) : n(_n), vis(_n + 1), match(_n + 1, -1) {}
7     bool dfs(int v) {
8         vis[v] = true;
9         for (int u : G[v])
10            if (match[u] == -1 || (!vis[match[u]] && dfs(match[u])))
11                {
12                    match[u] = v;
13                    return true;
14                }
15            return false;
16    }
17    int solve() {
18        int cnt = 0;
19        for (int i = 1; i <= n; i++) {
20            vis.assign(n + 1, 0);
21            if (dfs(i)) cnt++;
22        }
23        return cnt;
24    };

```

4.7 matching general

```

1 // O(V^3)
2 // if graph.match[i] == n, node i is not matched
3 // else, i is matched with graph.match[i]
4 struct Matching { // 0-based
5     int fa[N], pre[N], match[N], s[N], v[N], n, tk;
6     vector<int> g[N];
7     queue<int> q;
8     int Find(int u) {
9         return u == fa[u] ? u : fa[u] = Find(fa[u]);
10    }
11    int lca(int x, int y) {
12        tk++;
13        x = Find(x), y = Find(y);
14        for (; ; swap(x, y)) {
15            if (x != n) {
16                if (v[x] == tk) return x;
17                v[x] = tk;
18                x = Find(pre[match[x]]);
19            }
20        }
21    }
22    void blossom(int x, int y, int l) {
23        while (Find(x) != l) {
24            pre[x] = y, y = match[x];
25            if (s[y] == 1) q.push(y), s[y] = 0;
26            if (fa[x] == x) fa[x] = 1;
27            if (fa[y] == y) fa[y] = 1;
28            x = pre[y];
29        }
30    }
31    bool bfs(int r) {
32        for (int i = 0; i <= n; ++i) fa[i] = i, s[i] = -1;
33        while (!q.empty()) q.pop();
34        q.push(r);
35        s[r] = 0;

```

```

36 while (!q.empty()) {
37     int x = q.front(); q.pop();
38     for (int u : g[x]) {
39         if (s[u] == -1) {
40             pre[u] = x, s[u] = 1;
41             if (match[u] == n) {
42                 for (int a = u, b = x, last; b != n; a = last, b = pre[a])
43                     last = match[b], match[b] = a, match[a] = b;
44                 return true;
45             }
46             q.push(match[u]);
47             s[match[u]] = 0;
48         } else if (!s[u] && Find(u) != Find(x)) {
49             int l = lca(u, x);
50             blossom(x, u, l);
51             blossom(u, x, l);
52         }
53     }
54 }
55 return false;
56 }
57 int solve() {
58     int res = 0;
59     for (int x = 0; x < n; ++x) {
60         if (match[x] == n) res += bfs(x);
61     }
62     return res;
63 }
64 void init(int _n) {
65     n = _n, tk = 0;
66     for (int i = 0; i <= n; ++i) match[i] = pre[i] = n;
67     for (int i = 0; i < n; ++i) g[i].clear(), v[i] = 0;
68 }
69 void add_edge(int u, int v) {
70     g[u].push_back(v), g[v].push_back(u);
71 }
72 };

```

4.8 matching general weight

```

1 // O(V^3)
2 // if graph.match[i] == 0, node i is not matched
3 // else, i is matched with graph.match[i]
4 struct WeightGraph { // 1-based
5     static const int inf = INT_MAX;
6     static const int maxn = 514;
7     struct edge {
8         int u, v, w;
9         edge(){}
10        edge(int u, int v, int w): u(u), v(v), w(w) {}
11    };
12    int n, n_x;
13    edge g[maxn * 2][maxn * 2];
14    int lab[maxn * 2];
15    int match[maxn * 2], slack[maxn * 2], st[maxn * 2], pa[maxn * 2];
16    int flo_from[maxn * 2][maxn + 1], S[maxn * 2], vis[maxn * 2];
17    vector<int> flo[maxn * 2];
18    queue<int> q;
19    int e_delta(const edge &e) { return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
20    void update_slack(int u, int x) { if (!slack[x] || e_delta(g[u][x]) < e_delta(g[slack[x]][x])) slack[x] = u; }
21    void set_slack(int x) { slack[x] = 0; }
22    for (int u = 1; u <= n; ++u)
23        if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
24            update_slack(u, x);
25    }
26    void q_push(int x) {
27        if (x <= n) q.push(x);
28        else for (size_t i = 0; i < flo[x].size(); ++i) q_push(flo[x][i]);
29    }
30    void set_st(int x, int b) {
31        st[x] = b;
32        if (x > n) for (size_t i = 0; i < flo[x].size(); ++i)
33            set_st(flo[x][i], b);
34    }
35    int get_pr(int b, int xr) {
36        int pr = find(flo[b].begin(), flo[b].end(), xr) - flo[b].begin();
37        if (pr % 2 == 1) {
38            reverse(flo[b].begin() + 1, flo[b].end());
39            return (int)flo[b].size() - pr;
40        }
41        return pr;
42    }
43    void set_match(int u, int v) {
44        match[u] = g[u][v].v;
45        if (u <= n) return;
46        edge e = g[u][v];
47        int xr = flo_from[u][e.u], pr = get_pr(u, xr);

```

```

48     for (int i = 0; i < pr; ++i) set_match(flo[u][i], flo[u][i ^ 1]);
49     set_match(xr, v);
50     rotate(flo[u].begin(), flo[u].begin() + pr, flo[u].end());
51 }
52 void augment(int u, int v) {
53     for (; ) {
54         int xnv = st[match[u]];
55         set_match(u, v);
56         if (!xnv) return;
57         set_match(xnv, st[pa[xnv]]);
58         u = st[pa[xnv]], v = xnv;
59     }
60 }
61 int get_lca(int u, int v) {
62     static int t = 0;
63     for (++t; u || v; swap(u, v)) {
64         if (u == 0) continue;
65         if (vis[u] == t) return u;
66         vis[u] = t;
67         u = st[match[u]];
68         if (u) u = st[pa[u]];
69     }
70     return 0;
71 }
72 void add_blossom(int u, int lca, int v) {
73     int b = n + 1;
74     while (b <= n_x && st[b]) ++b;
75     if (b > n_x) ++n_x;
76     lab[b] = 0, S[b] = 0;
77     match[b] = match[lca];
78     flo[b].clear();
79     flo[b].push_back(lca);
80     for (int x = u, y; x != lca; x = st[pa[y]])
81         flo[b].push_back(x), flo[b].push_back(y = st[match[x]]),
82         q_push(y);
83     reverse(flo[b].begin() + 1, flo[b].end());
84     for (int x = v, y; x != lca; x = st[pa[y]])
85         flo[b].push_back(x), flo[b].push_back(y = st[match[x]]),
86         q_push(y);
87     set_st(b, b);
88     for (int x = 1; x <= n_x; ++x) g[b][x].w = g[x][b].w = 0;
89     for (int x = 1; x <= n; ++x) flo_from[b][x] = 0;
90     for (size_t i = 0; i < flo[b].size(); ++i) {
91         int xs = flo[b][i];
92         for (int x = 1; x <= n_x; ++x)
93             if (g[b][x].w == 0 || e_delta(g[xs][x]) < e_delta(g[b][x]))
94                 g[b][x] = g[xs][x], g[x][b] = g[x][xs];
95         for (int x = 1; x <= n; ++x)
96             if (flo_from[xs][x]) flo_from[b][x] = xs;
97     }
98     set_slack(b);
99 }
100 void expand_blossom(int b) {
101     for (size_t i = 0; i < flo[b].size(); ++i)
102         set_st(flo[b][i], flo[b][i]);
103     int xr = flo_from[b][g[b][pa[b]].u], pr = get_pr(b, xr);
104     for (int i = 0; i < pr; ++i) {
105         int xs = flo[b][i], xns = flo[b][i + 1];
106         pa[xs] = g[xns][xs].u;
107         S[xs] = 1, S[xns] = 0;
108         slack[xs] = 0, set_slack(xns);
109         q_push(xns);
110     }
111     S[xr] = 1, pa[xr] = pa[b];
112     for (size_t i = pr + 1; i < flo[b].size(); ++i) {
113         int xs = flo[b][i];
114         S[xs] = -1, set_slack(xs);
115     }
116     st[b] = 0;
117 }
118 bool on_found_edge(const edge &e) {
119     int u = st[e.u], v = st[e.v];
120     if (S[v] == -1) {
121         pa[v] = e.u, S[v] = 1;
122         int nu = st[match[v]];
123         slack[v] = slack[nu] = 0;
124         S[nu] = 0, q_push(nu);
125     } else if (S[v] == 0) {
126         int lca = get_lca(u, v);
127         if (!lca)
128             return augment(u, v), augment(v, u), true;
129         else
130             add_blossom(u, lca, v);
131     }
132     return false;
133 }
134 bool matching() {
135     memset(S + 1, -1, sizeof(int) * n_x);
136     memset(slack + 1, 0, sizeof(int) * n_x);
137     q = queue<int>();
138     for (int x = 1; x <= n_x; ++x)
139         if (st[x] == x && !match[x])
140             pa[x] = 0, S[x] = 0, q_push(x);
141     if (q.empty()) return false;
142     for (; ) {

```

```

141 while (q.size()) {
142     int u = q.front(); q.pop();
143     if (S[st[u]] == 1) continue;
144     for (int v = 1; v <= n; ++v)
145         if (g[u][v].w > 0 && st[u] != st[v]) {
146             if (e_delta(g[u][v]) == 0) {
147                 if (on_found_edge(g[u][v]))
148                     return true;
149             } else
150                 update_slack(u, st[v]);
151         }
152     }
153     int d = inf;
154     for (int b = n + 1; b <= n_x; ++b)
155         if (st[b] == b && S[b] == 1) d = min(d, lab[b] / 2);
156     for (int x = 1; x <= n_x; ++x)
157         if (st[x] == x && slack[x]) {
158             if (S[x] == -1) d = min(d, e_delta(g[slack[x]][x]));
159             else if (S[x] == 0) d = min(d, e_delta(g[slack[x]][x]
160                 ) / 2);
161         }
162     for (int u = 1; u <= n; ++u) {
163         if (S[st[u]] == 0) {
164             if (lab[u] <= d) return 0;
165             lab[u] -= d;
166         } else if (S[st[u]] == 1) lab[u] += d;
167     }
168     for (int b = n + 1; b <= n_x; ++b)
169         if (st[b] == b) {
170             if (S[st[b]] == 0) lab[b] += d * 2;
171             else if (S[st[b]] == 1) lab[b] -= d * 2;
172         }
173     q = queue<int>();
174     for (int x = 1; x <= n_x; ++x)
175         if (st[x] == x && slack[x] && st[slack[x]] != x &&
176             e_delta(g[slack[x]][x]) == 0)
177             if (on_found_edge(g[slack[x]][x]))
178                 return true;
179     for (int b = n + 1; b <= n_x; ++b)
180         if (st[b] == b && S[b] == 1 && lab[b] == 0)
181             expand_blossom(b);
182     }
183     return false;
184 }
185 pair<long long, int> solve() {
186     memset(match + 1, 0, sizeof(int) * n);
187     n_x = n;
188     int n_matches = 0;
189     long long tot_weight = 0;
190     for (int u = 0; u <= n; ++u) st[u] = u, flo[u].clear();
191     int w_max = 0;
192     for (int u = 1; u <= n; ++u)
193         for (int v = 1; v <= n; ++v) {
194             flo_from[u][v] = (u == v ? u : 0);
195             w_max = max(w_max, g[u][v].w);
196         }
197     for (int u = 1; u <= n; ++u) lab[u] = w_max;
198     while (matching()) ++n_matches;
199     for (int u = 1; u <= n; ++u)
200         if (match[u] && match[u] < u)
201             tot_weight += g[u][match[u]].w;
202     return make_pair(tot_weight, n_matches);
203 }
204 void add_edge(int ui, int vi, int wi) {
205     g[ui][vi].w = g[vi][ui].w = wi;
206 }
207 void init(int _n) {
208     n = _n;
209     for (int u = 1; u <= n; ++u)
210         for (int v = 1; v <= n; ++v)
211             g[u][v] = edge(u, v, 0);
212 }

```

4.9 prim

```

1 const int N = 2005;
2 int dis[N][N];
3 int distfromtree[N];
4 for (int i = 1; i <= n; i++)
5     distfromtree[i] = dis[1][i];
6 int res = 0;
7 for (int i = 2; i <= n; i++) {
8     int mindist = INT_MAX, next = -1;
9     for (int j = 1; j <= n; j++) {
10         if (distfromtree[j] != 0 && distfromtree[j] < mindist) {
11             mindist = distfromtree[j];
12             next = j;
13         }
14     }
15     res += mindist;
16     for (int j = 1; j <= n; j++)
17         distfromtree[j] = min(distfromtree[j], dis[next][j]);
18 }

```

4.10 scc

```

1 const int N = 200005;
2 vector<int> G[N], rev_G[N], order;
3 bool vis[N]; int scc[N];
4 void rev_dfs(int v) {
5     vis[v] = true;
6     for (int u : rev_G[v])
7         if (!vis[u])
8             rev_dfs(u);
9     order.push_back(v);
10 }
11 void dfs(int v, int block) {
12     scc[v] = block;
13     for (int u : G[v])
14         if (scc[u] == -1)
15             dfs(u, block);
16 }
17 void kosaraju(int n) {
18     for (int i = 1; i <= n; i++)
19         if (!vis[i])
20             rev_dfs(i);
21     int scc_cnt = 0;
22     fill(scc, scc + N, -1);
23     for (int i = n - 1; i >= 0; i--)
24         if (scc[order[i]] == -1)
25             dfs(order[i], ++scc_cnt);
26 }

```

4.11 spfa

```

1 vector<int> G[N];
2 int spfa(int n, int s, int t) {
3     vector<int> dis(n, INT_MAX), cnt(n);
4     vector<bool> inq(n);
5     queue<int> q;
6     q.push(s);
7     dis[s] = 0;
8     inq[s] = true;
9     while (q.size()) {
10         int u = q.front();
11         q.pop();
12         cnt[u]++;
13         if (cnt[u] == n) // negative cycle
14             return INT_MAX;
15         inq[u] = false;
16         for (auto [v, w] : G[u]) {
17             if (dis[u] + w < dis[v]) {
18                 dis[v] = dis[u] + w;
19                 if (!inq[v]) {
20                     inq[v] = true;
21                     q.push(v);
22                 }
23             }
24         }
25     }
26     return dis[t];
27 }

```

5 math

5.1 big number

```

1 void init(string &a, string &b) {
2     while (a.size() < b.size()) a = '0' + a;
3     while (a.size() > b.size()) b = '0' + b;
4 }
5 int Compare(string &a, string &b) {
6     if (a < b) {
7         swap(a, b); return -1;
8     }
9     return 1;
10 }
11 bool del(string &a) {
12     if (a[0] == '0') {
13         a.erase(0, 1); return true;
14     }
15     return false;
16 }
17 void del_all_zero(string &a) {
18     while (del(a)) ;
19 }
20 string add(string a, string b) {
21     init(a, b);
22     a = '0' + a; b = '0' + b;
23     for (int i = a.size() - 1; i >= 0; i--) {

```

```

24     int p = a[i] - '0', q = b[i] - '0';
25     if (p + q >= 10) {
26         a[i - 1] = '0' + (a[i - 1] - '0' + 1);
27         a[i] = '0' + (p + q - 10);
28     } else {
29         a[i] = '0' + (p + q);
30     }
31 }
32 del(a);
33 return a;
34 }
35 string sub(string a, string b) {
36     init(a, b);
37     if (a == b) return "0";
38     int flag = Compare(a, b);
39     for (int i = a.size() - 1; i >= 0; i--) {
40         int p = a[i] - '0', q = b[i] - '0';
41         if (p < q) {
42             a[i - 1] = '0' + (a[i - 1] - '0' - 1);
43             a[i] = '0' + (p - q + 10);
44         } else {
45             a[i] = '0' + (p - q);
46         }
47     }
48     del_all_zero(a);
49     if (flag == -1) a = "-" + a;
50     return a;
51 }
52 string mul(string a, string b) {
53     string res = "0";
54     init(a, b);
55     Compare(a, b);
56     del_all_zero(b);
57     for (int i = b.size() - 1; i >= 0; i--) {
58         int x = b[i] - '0';
59         if (i != b.size() - 1) a = a + '0';
60         for (int i = 0; i < x; i++)
61             res = add(a, res);
62     }
63     return res;
64 }
65 string div(string a, string b) {
66     init(a, b);
67     if (a < b) return "0";
68     del_all_zero(b);
69     string res = "0", restmp = "1", tmp = b;
70     for (int i = 0; i < (a.size() - b.size()); i++) {
71         restmp += '0';
72         tmp += '0';
73     }
74     init(a, b);
75     while (a >= b) {
76         init(a, tmp);
77         if (a >= tmp) {
78             a = sub(a, tmp);
79             res = add(res, restmp);
80         } else {
81             restmp.erase(restmp.size() - 1);
82             tmp.erase(tmp.size() - 1);
83         }
84     }
85     init(a, b);
86     return res;
87 }

```

5.2 burnside lemma

$$\sum_{k=1}^n \frac{c(k)}{n}$$

- n : 有多少種置換方式 (例如: 旋轉方式)
- $c(k)$: 所有可能中, 經過 k 次旋轉後, 仍不會和別人相同方式的數量

5.3 catalan

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

5.4 chinese remainder theorem coprime

```

1 vector<int> a, m;
2 int extgcd(int a, int b, int& x, int& y) {
3     if (b == 0) {
4         x = 1, y = 0;
5         return a;
6     }
7     int ret = extgcd(b, a % b, y, x);
8     y -= a / b * x;
9     return ret;

```

```

10 }
11 // n = 有幾個式子 · 求解  $x \equiv a_i \pmod{m_i}$ 
12 int CRT(int n, vector<int>& a, vector<int>& m) {
13     int p = 1, ans = 0;
14     vector<int> M(n), inv_M(n);
15     for (int i = 0; i < n; i++)
16         p *= m[i];
17     for (int i = 0; i < n; i++) {
18         M[i] = p / m[i];
19         extgcd(M[i], m[i], inv_M[i], tmp);
20         ans += a[i] * inv_M[i] * M[i];
21         ans %= p;
22     }
23     return (ans % p + p) % p;
24 }

```

5.5 chinese remainder theorem coprime

n = 有幾個式子 · 求解 $x \equiv a_i \pmod{m_i}$

5.6 chinese remainder theorem not coprime

```

1 int extgcd(int a, int b, int& x, int& y) {
2     if (b == 0) {
3         x = 1, y = 0;
4         return a;
5     }
6     int ret = extgcd(b, a % b, y, x);
7     y -= a / b * x;
8     return ret;
9 }
10 // 對於方程組的式子兩兩求解 · return {是否有解, {a, m}}
11 pair<bool, pair<int, int>> CRT(int a1, int m1, int a2, int m2)
12 {
13     int g = __gcd(m1, m2);
14     if ((a2 - a1) % g != 0)
15         return {0, {-1, -1}};
16     int x, y;
17     extgcd(m1, m2, x, y);
18     x = (a2 - a1) * x / g; // 兩者不能相反
19     a1 = x * m1 + a1;
20     m1 = m1 * m2 / g;
21     a1 = (a1 % m1 + m1) % m1;
22     return {1, {a1, m1}};
23 }

```

5.7 count prime

```

1 // Count the number of primes not greater than N ( $N \leq 10^{11}$ )
2 int primeCount(const int n) {
3     if (n <= 1) { return 0; }
4     if (n == 2) { return 1; }
5     const int v = sqrt1(n);
6     int s = (v + 1) / 2;
7     vector<int> smalls(s), roughs(s), skip(v + 1), larges(s);
8     iota(smalls.begin(), smalls.end(), 0);
9     for (int i = 0; i < s; i++) {
10         roughs[i] = 2 * i + 1;
11         larges[i] = (n / roughs[i] - 1) / 2;
12     }
13     const auto half = [](int n) -> int { return (n - 1) >> 1; };
14     int pc = 0;
15     for (int p = 3; p <= v; p += 2) {
16         if (skip[p]) { continue; }
17         int q = p * p;
18         if (1LL * q * q > n) { break; }
19         skip[p] = true;
20         for (int i = q; i <= v; i += 2 * p) skip[i] = true;
21         int ns = 0;
22         for (int k = 0; k < s; k++) {
23             int i = roughs[k];
24             if (skip[i]) { continue; }
25             int d = 1LL * i * p;
26             larges[ns] = larges[k] - (d <= v ? larges[smalls[d / 2] - pc] : smalls[half(n / d)]) + pc;
27             roughs[ns++] = i;
28         }
29         s = ns;
30         for (int i = half(v), j = v / p - 1 | 1; j >= p; j -= 2) {
31             int c = smalls[j / 2] - pc;
32             for (int e = j * p / 2; i >= e; i--) { smalls[i] -= c; }
33         }
34         pc++;
35     }
36     larges[0] += 1LL * (s + 2 * (pc - 1)) * (s - 1) / 2;
37     for (int k = 1; k < s; k++) { larges[0] -= larges[k]; }
38     for (int l = 1; l < s; l++) {
39         int q = roughs[l];

```

```

40     int M = n / q;
41     int e = smalls[half(M / q)] - pc;
42     if (e <= 1) { break; }
43     int t = 0;
44     for (int k = 1 + 1; k <= e; k++) { t += smalls[half(M /
45         roughs[k])]; }
46     larges[0] += t - 1LL * (e - 1) * (pc + 1 - 1);
47     return larges[0] + 1;
48 }

```

5.8 det

```

1 ll Det(vector<vector<ll>> a) {
2     int n = a.size();
3     ll det = 1;
4     for (int i = 0; i < n; ++i) {
5         if (!a[i][i]) {
6             det = -det;
7             if (det < 0) det += mod;
8             for (int j = i + 1; j < n; ++j) if (a[j][i]) {
9                 swap(a[j], a[i]);
10                break;
11            }
12            if (!a[i][i]) return 0;
13        }
14        det = det * a[i][i] % mod;
15        ll mul = mpow(a[i][i], mod - 2);
16        for (int j = 0; j < n; ++j)
17            a[i][j] = a[i][j] * mul % mod;
18        for (int j = 0; j < n; ++j) if (i ^ j) {
19            ll mul = a[j][i];
20            for (int k = 0; k < n; ++k) {
21                a[j][k] -= a[i][k] * mul % mod;
22                if (a[j][k] < 0) a[j][k] += mod;
23            }
24        }
25    }
26    return det;
27 }

```

5.9 extgcd

```

1 // a * p.first + b * p.second = gcd(a, b)
2 pair<ll, ll> extgcd(ll a, ll b) {
3     pair<ll, ll> res, tmp;
4     ll f = 1, g = 1;
5     if (a < 0) a *= -1, f *= -1;
6     if (b < 0) b *= -1, g *= -1;
7     if (b == 0) return {f, 0};
8     tmp = extgcd(b, a % b);
9     res.first = tmp.second * f;
10    res.second = (tmp.first - tmp.second * (a / b)) * g;
11    return res;
12 }

```

5.10 factorize

```

1 using i64 = long long;
2 using i128 = __int128_t;
3 struct Factorize { // 質因數分解 N <=
4     i64 fmul(i64 a, i64 b, i64 p) {
5         return (i128)a * b % p;
6     }
7     i64 fpow(i64 a, i64 b, i64 p) {
8         i64 res = 1;
9         for (; b; b >>= 1, a = fmul(a, a, p))
10             if (b & 1) res = fmul(res, a, p);
11         return res;
12     }
13     bool check(i64 a, i64 u, i64 n, int t) {
14         a = fpow(a, u, n);
15         if (a == 0 || a == 1 || a == n - 1)
16             return true;
17         for (int i = 0; i < t; i++) {
18             a = fmul(a, a, n);
19             if (a == 1) return false;
20             if (a == n - 1) return true;
21         }
22         return false;
23     };
24     bool isPrime(i64 n) {
25         constexpr array<i64, 7> magic{2, 235, 9375, 28178, 450775,
26             9780504, 1795265022};
27         // for int: {2, 7, 61}
28         if (n < 2) return false;
29         if (n % 2 == 0) return n == 2;

```

```

29         i64 u = n - 1;
30         int t = 0;
31         while (u % 2 == 0) u >>= 1, t++;
32         for (auto v : magic) if (!check(v, u, n, t)) return false;
33         return true;
34     }
35     i64 PollardRho(i64 n) { // return non-trivial factor of n
36         if (n % 2 == 0) return 2;
37         i64 x = 2, y = 2, d = 1, p = 1;
38         auto f = [](i64 x, i64 n, i64 p) -> i64 {
39             return ((i128)x * x % n + p) % n;
40         };
41         while (true) {
42             x = f(x, n, p);
43             y = f(f(y, n, p), n, p);
44             d = gcd(abs(x - y), n);
45             if (d != n && d != 1) return d;
46             if (d == n) ++p;
47         }
48     }
49     i64 primeFactor(i64 n) {
50         return isPrime(n) ? n : primeFactor(PollardRho(n));
51     }
52 } ftr;
53 void solve() {
54     i64 n;
55     cin >> n;
56     vector<i64> ans;
57     while (n > 1) {
58         i64 p = ftr.primeFactor(n);
59         do {
60             ans.push_back(p);
61             n /= p;
62         } while (n % p == 0);
63     }
64     sort(ans.begin(), ans.end());
65     cout << ans.size() << ' ';
66     for (int i : ans) cout << i << ' ';
67     cout << '\n';
68 }

```

5.11 fraction

```

1 struct frac {
2     int a, b;
3     frac(int _a = 0, int _b = 1) : a(_a), b(_b) {
4         int g = __gcd(a, b);
5         a /= g; b /= g;
6         if (b < 0) { a *= -1; b *= -1; }
7     }
8 };
9 frac operator+(frac x, frac y) { return frac(x.a * y.b + y.a *
10     x.b, x.b * y.b); }
11 frac operator-(frac x, frac y) { return frac(x.a * y.b - y.a *
12     x.b, x.b * y.b); }
13 frac operator*(frac x, frac y) { return frac(x.a * y.a, x.b * y
14     .b); }
15 frac operator/(frac x, frac y) { return frac(x.a * y.b, x.b * y
16     .a); }
17 bool operator>(frac x, frac y) { return x.a * y.b > y.a * x.b;
18 }
19 bool operator<(frac x, frac y) { return x.a * y.b < y.a * x.b;
20 }
21 bool operator>=(frac x, frac y) { return x.a * y.b >= y.a * x.b
22 ; }
23 bool operator<=(frac x, frac y) { return x.a * y.b <= y.a * x.b
24 ; }
25 bool operator==(frac x, frac y) { return x.a * y.b == y.a * x.b
26 ; }
27 frac abs(frac x) { return frac(abs(x.a), abs(x.b)); }
28 ostream& operator<<(ostream &os, const frac& x) { os << x.a <<
29     "/" << x.b; return os; }

```

5.12 gaussian elimination

```

1 #define int long long
2 const int N = 25;
3 frac co[N][N];
4 // SIZE = n * (n + 1)
5 // a_1 x_1 + a_2 x_2 + ... + a_n x_n = a_{n+1}
6 void gaussian(int n) {
7     for (int i = 1; i <= n; i++) {
8         if (co[i][i].a == 0 && co[i][i].b == 1) { // swap
9             int swapid = -1;
10            for (int id = i + 1; id <= n; id++) {
11                if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
12                    swapid = id; break;
13                }
14            }
15            if (swapid == -1)
16                continue;

```

```

17     else {
18         for (int j = 1; j <= n + 1; j++)
19             swap(co[i][j], co[swapid][j]);
20     }
21 }
22 for (int id = i + 1; id <= n; id++) {
23     if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
24         frac mul = co[id][i] / co[i][i];
25         for (int j = 1; j <= n + 1; j++)
26             co[id][j] = co[id][j] - co[i][j] * mul;
27     }
28 }
29 }
30
31 for (int i = n; i >= 1; i--) {
32     if (! (co[i][i].a == 0 && co[i][i].b == 1)) {
33         co[i][n + 1] = co[i][n + 1] / co[i][i];
34         co[i][i] = {1, 1};
35     } else
36         continue;
37     for (int id = 1; id < i; id++) {
38         if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
39             frac mul = co[id][i] / co[i][i];
40             for (int j = 1; j <= n + 1; j++)
41                 co[id][j] = co[id][j] - co[i][j] * mul;
42         }
43     }
44 }
45 bool check_inf = false, check_no = false;
46 for (int i = 1; i <= n; i++) {
47     bool allzro = true;
48     for (int j = 1; j <= n; j++)
49         if (! (co[i][j].a == 0 && co[i][j].b == 1))
50             allzro = false;
51     if (allzro) {
52         if (! (co[i][n + 1].a == 0 && co[i][n + 1].b == 1))
53             check_no = true;
54     } else
55         check_inf = true;
56 }
57 }
58 if (check_no) {
59     cout << "no solutions\n";
60 } else if (check_inf) {
61     cout << "infinite many solutions\n";
62 } else {
63     for (int i = 1; i <= n; i++)
64         cout << "X" << i << " = " << co[i][n + 1] << '\n';
65 }
66 }

```

5.13 josephus

```

1 // 有 n 個人，第偶數個報數的人被刪掉，問第 k 個被踢掉的是誰
2 int solve(int n, int k) {
3     if (n == 1) return 1;
4     if (k <= (n + 1) / 2) {
5         if (2 * k > n)
6             return 2 * k % n;
7         else
8             return 2 * k;
9     } else {
10         int res = solve(n / 2, k - (n + 1) / 2);
11         if (n & 1)
12             return 2 * res + 1;
13         else
14             return 2 * res - 1;
15     }
16 }

```

5.14 lucas

```

1 // 對於很大的  $C_n^m$  對質數 p 取模，只要 p 不大就可以用
2 int Lucas(int n, int m, int p) {
3     if (m == 0) return 1;
4     return (C(n % p, m % p, p) * Lucas(n / p, m / p, p) % p);
5 }

```

5.15 math block

```

1 // the sum of floor(n / i) for i = 1 ~ n
2 int f(int n) {
3     int ans = 0;
4     int l = 1, r = 0;
5     while (l <= n) {
6         r = n / (n / l);
7         ans += (r - l + 1) * (n / l);

```

```

8         l = r + 1;
9     }
10     return ans;
11 }
12 // the sum of ceil(n / i) for i = 1 ~ n
13 int f(int n) {
14     int ans = 0;
15     int l = 1, r = 0;
16     while (l < n) {
17         r = (n - 1) / ((n - 1) / l);
18         ans += (r - l + 1) * ((n + l - 1) / l);
19         l = r + 1;
20     }
21     if (l == n) ans += (n + l - 1) / l;
22     return ans;
23 }

```

5.16 miller rabin

```

1 struct Miller_Rabin {
2     static constexpr uint32_t ws32[3] = {2, 7, 61};
3     static constexpr uint64_t ws64[7] = {2, 325, 9375, 28178,
4         450775, 9780504, 1795265022};
5     template <class uint>
6     static constexpr uint fast_pow(uint b, uint p, uint mod) {
7         using Uint = conditional_t<is_same_v<uint, uint32_t>,
8             uint64_t, __uint128_t>;
9         uint ret{1};
10        for (; p; p >>= 1) {
11            if (p & 1) ret = Uint{ret} * b % mod;
12            b = Uint{b} * b % mod;
13        }
14        return ret;
15    }
16    template <class uint, class = enable_if_t<is_unsigned_v<uint>
17        >>>
18    static constexpr bool is_prime(uint n) {
19        const auto& witness = []() -> const auto& {
20            if constexpr (is_same_v<uint, uint32_t>) return ws32;
21            else return ws64;
22        }();
23        using Uint = conditional_t<is_same_v<uint, uint32_t>,
24            uint64_t, __uint128_t>;
25        if (n < 3 || !(n & 1)) return n == 2;
26        auto u = n - 1, t = (uint) __builtin_ctzll(u);
27        u >>= t;
28        for (auto x : witness) {
29            auto a = x % n;
30            if (a == 0 || a == 1 || a == n - 1) continue;
31            auto v = fast_pow(a, u, n);
32            uint i = 0;
33            for (; v != 1 && v != n - 1 && i < t; i++) v = Uint{v} *
34                v % n;
35            if (v != n - 1 && i > 0) return false;
36        }
37        return true;
38    }
39    template <class I, class = enable_if_t<is_integral_v<I>>>
40    constexpr bool operator()(I x) const {
41        if constexpr (is_signed_v<I>) if (x < 0) return false;
42        using U = make_unsigned_t<I>;
43        return is_prime((U)x);
44    }
45 } is_prime;

```

5.17 phi

```

1 // phi(n) = 小於 n 並與 n 互質的正整數數量。
2 // O(sqrt(n)) · 回傳 phi(n)
3 int phi(int n) {
4     int ret = n;
5     for (int i = 2; i * i <= n; i++) {
6         if (n % i == 0) {
7             while (n % i == 0) n /= i;
8             ret = ret * (i - 1) / i;
9         }
10    }
11    if (n > 1) ret = ret * (n - 1) / n;
12    return ret;
13 }
14 // O(n log n) · 回傳 1 ~ n 的 phi 值
15 vector<int> phi_1_to_n(int n) {
16     vector<int> phi(n + 1);
17     phi[0] = 0; phi[1] = 1;
18     for (int i = 2; i <= n; i++) phi[i] = i - 1;
19     for (int i = 2; i <= n; i++)
20         for (int j = 2 * i; j <= n; j += i) // 枚舉所有倍數
21             phi[j] -= phi[i];
22     return phi;
23 }

```

5.18 phi lemma

若 a, m 互質，則：

$$a^n \bmod m = a^{n \bmod \varphi(m)} \bmod m$$

若 a, m 可能是任何數，則：

$$a^{\varphi(m) + [n \bmod \varphi(m)]} \bmod m$$

5.19 primes er

```
1 // Time Complexity : O(nLogLogn)
2 const int N = 200005;
3 bool is_prime[N];
4 vector<int> primes;
5 void build() {
6     fill(is_prime, is_prime + N, true);
7     is_prime[0] = is_prime[1] = false;
8     for (int i = 2; i <= 200000; i++) {
9         if (is_prime[i]) {
10             primes.push_back(i);
11             if (1ll * i * i <= 200000)
12                 for (int j = i * i; j <= 200000; j += i)
13                     is_prime[j] = false;
14         }
15     }
16 }
```

5.20 primes linear

```
1 // Time Complexity : O(n)
2 // if lpf(i) = i, then it means that i is a prime.
3 // else lpf(i) is the smallest prime factor.
4 // The 199999-th of the prime is 2750131
5 bool is_prime[2750135];
6 int lpf[2750135];
7 vector<int> primes;
8 void init() {
9     fill(is_prime, is_prime + 2750135, true);
10    for (int i = 2; i <= 2750131; i++) {
11        if (is_prime[i]) {
12            primes.push_back(i);
13            lpf[i] = i;
14        }
15        for (int p : primes) {
16            if (p * i > 2750131) break;
17            is_prime[p * i] = false;
18            lpf[p * i] = p;
19            if (i % p == 0) break;
20        }
21    }
22 }
```

5.21 two equations

$$\begin{cases} ax + by = c \\ dx + ey = f \end{cases} = \begin{cases} x = \frac{ed-bf}{ad-bc} \\ y = \frac{af-ec}{ad-bc} \end{cases}$$

若 $x = \frac{0}{0}$ 且 $y = \frac{0}{0}$ ，則代表無限多組解。若 $x = \frac{*}{0}$ 且 $y = \frac{*}{0}$ ，則代表無解。

6 other

6.1 aliens dp

```
1 const int N = 2000005;
2 int n, k;
3 int a[N];
4 pii dp0[N], dp1[N];
5 pii calc(int c) {
6     dp0[1] = {0, 0}; dp1[1] = {c - a[1], 1};
7     for (int i = 2; i <= n; i++) {
8         dp0[i] = max(dp0[i - 1], dp1[i - 1] + make_pair(+a[i], 0));
9         dp1[i] = max(dp1[i - 1], dp0[i - 1] + make_pair(-a[i] + c, 1));
10    }
11    return dp0[n];
12 }
13 void solve() {
14     pii p = calc(0);
15     if (p.S <= k) {
16         cout << p.F << "\n";
17         return;
18     }
```

```
18 }
19 int l = -1e5, r = 1e6; pii res;
20 while (l + 1 < r) {
21     int mid = (l + r) >> 1;
22     res = calc(mid);
23     if (res.S == k) {
24         cout << res.F - mid * k << "\n";
25         return;
26     } else if (res.S > k)
27         r = mid;
28     else
29         l = mid;
30 }
31 pii p1 = calc(l), p2 = calc(r);
32 int dp_1 = p1.F - l * p1.S;
33 int dp_2 = p2.F - r * p2.S;
34 cout << (dp_1 + (dp_2 - dp_1) * (k - p1.S) / (p2.S - p1.S))
35     << "\n";
36 }
```

6.2 all LCS

```
1 void all_lcs(string s, string t) { // 0-base
2     vector<int> h(sz(t));
3     iota(all(h), 0);
4     for (int a = 0; a < sz(s); ++a) {
5         int v = -1;
6         for (int c = 0; c < sz(t); ++c)
7             if (s[a] == t[c] || h[c] < v)
8                 swap(h[c], v);
9         // LCS(s[0, a], t[b, c]) =
10         // c - b + 1 - sum([h[i] >= b] | i <= c)
11         // h[i] might become -1 !!
12     }
13 }
```

6.3 dc dp

```
1 const int N = 3005;
2 int n, k, a[N], pre[N], dp[N][N];
3 int cost(int l, int r) {}
4 void dc(int j, int l, int r, int best_l, int best_r) {
5     if (l > r) return;
6     int mid = (l + r) >> 1, pos = 1;
7     for (int i = best_l; i <= best_r; i++) {
8         if (dp[i][j - 1] + cost(i + 1, mid) < dp[mid][j]) {
9             pos = i;
10            dp[mid][j] = dp[i][j - 1] + cost(i + 1, mid);
11        }
12    }
13    dc(j, l, mid - 1, best_l, pos);
14    dc(j, mid + 1, r, pos, best_r);
15 }
16 void solve() {
17     for (int i = 1; i <= n; i++)
18         dp[i][1] = pre[i] * pre[i];
19     for (int j = 2; j <= k; j++)
20         dc(j, j, n, j - 1, n);
21 }
```

6.4 digit dp

```
1 // 原題：計算 [L, r] 中兩兩相鄰數字不同的數字數量
2 // 注意邊界
3 int dp[20][10]; // i 位數，尾數是 j
4 void init() {
5     for (int j = 0; j <= 9; j++) dp[1][j] = 1;
6     for (int i = 2; i <= 18; i++)
7         for (int now = 0; now <= 9; now++) // 這個位數的數字
8             for (int pre = 0; pre <= 9; pre++) // 上個位數的數字
9                 if (now != pre)
10                     dp[i][now] += dp[i - 1][pre];
11 }
12 int calc(int x) { // 小於等於 x 的數量
13     if (x == 0) return 0;
14     int arr[20] = {}, len = 0;
15     while (x > 0) arr[++len] = x % 10, x /= 10;
16     // 計算 Len 位數的數量
17     int cnt = 0, pre = -1;
18     for (int idx = len; idx >= 1; idx--) {
19         for (int now = (idx == len); now < arr[idx]; now++)
20             if (now != pre)
21                 cnt += dp[idx][now];
22         if (arr[idx] == pre) break;
23         pre = arr[idx];
24     }
25     if (idx == 1) cnt++; // 特判
```

```

25 }
26 // 計算 小於 Len 位數 的數量
27 for (int i = 1; i < len; i++)
28     for (int j = 1; j <= 9; j++)
29         cnt += dp[i][j];
30 return cnt;
31 }

```

6.5 geometry

```

1 double eps = 0.000001;
2 pdd operator + (pdd a, pdd b) { return {a.X + b.X, a.Y + b.Y}; }
3 pdd operator - (pdd a, pdd b) { return {a.X - b.X, a.Y - b.Y}; }
4 double dot(pdd a, pdd b) { return a.X * b.X + a.Y * b.Y; }
5 double cross(pdd a, pdd b) { return a.X * b.Y - a.Y * b.X; }
6 int sign(double x) { return (fabs(x) < eps ? 0 : (x > 0 ? 1 : -1)); }
7 int ori(pdd a, pdd b, pdd c) { return sign(cross(b - a, c - a)); }
8 // c is between a and b
9 bool btw(pdd a, pdd b, pdd c) {
10     return (sign(cross(a - c, b - c)) == 0 && sign(dot(a - c, b - c)) <= 0);
11 }
12 // ab is touch cd
13 bool banana(pdd a, pdd b, pdd c, pdd d) {
14     int a123 = ori(a, b, c), a124 = ori(a, b, d);
15     int a341 = ori(c, d, a), a342 = ori(c, d, b);
16     if (btw(a, b, c) || btw(a, b, d) || btw(c, d, a) || btw(c, d, b))
17         return true;
18     return (a123 * a124 < 0 && a341 * a342 < 0);
19 }
20 vector<pdd> convex_hull(vector<pdd> pts) {
21     sort(pts.begin(), pts.end());
22     pts.resize(unique(pts.begin(), pts.end()) - pts.begin());
23     vector<pdd> hull(1, pts[0]);
24     for (int t = 0; t < 2; t++) {
25         int sz = hull.size();
26         for (int i = 1; i < (int)pts.size(); i++) {
27             while ((int)hull.size() > sz && ori(hull[(int)hull.size() - 2], hull.back(), pts[i]) < 0)
28                 hull.pop_back();
29             hull.push_back(pts[i]);
30         }
31         reverse(pts.begin(), pts.end());
32     }
33     hull.pop_back();
34     return hull;
35 }
36 double area(vector<pdd> pt) {
37     double cnt = 0.0;
38     for (int i = 0; i < (int)pt.size() - 1; i++)
39         cnt += cross(pt[i], pt[i + 1]);
40     cnt += cross(pt[(int)pt.size() - 1], pt[0]);
41     return fabs(cnt);
42 }
43 void solve() {
44     int n;
45     cin >> n;
46     vector<pdd> pts;
47     pts.resize(n);
48     for (int i = 0; i < n; i++)
49         cin >> pts[i].X >> pts[i].Y;
50     int ans = area(convex_hull(pts));
51     if (ans % 2 == 1)
52         cout << ans / 2 << ".5\n";
53     else
54         cout << ans / 2 << ".0\n";
55 }

```

6.6 line container

```

1 // only works for integer coordinates!! maintain max
2 struct Line {
3     mutable ll a, b, p;
4     bool operator<(const Line &rhs) const { return a < rhs.a; }
5     bool operator<(ll x) const { return p < x; }
6 };
7 struct DynamicHull : multiset<Line, less<>> {
8     static const ll kInf = 1e18;
9     ll Div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }
10    bool isect(iterator x, iterator y) {
11        if (y == end()) { x->p = kInf; return 0; }
12        if (x->a == y->a) x->p = x->b > y->b ? kInf : -kInf;
13        else x->p = Div(y->b - x->b, x->a - y->a);
14        return x->p >= y->p;
15    }

```

```

16 void addline(ll a, ll b) { // ax + b
17     auto z = insert({a, b, 0}), y = z++, x = y;
18     while (isect(y, z)) z = erase(z);
19     if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
20     while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
21 }
22 ll query(ll x) {
23     auto l = *lower_bound(x);
24     return l.a * x + l.b;
25 }
26 };

```

6.7 mo

```

1 struct MoSolver {
2     struct query {
3         int l, r, id;
4         bool operator < (const query &o) {
5             if (1 / C == 0.1 / C) return (1 / C) & 1 ? r > o.r : r < o.r;
6             return 1 / C < 0.1 / C;
7         }
8     };
9     int cur_ans;
10    vector<int> ans;
11    void add(int x) {
12        // do something
13    }
14    void sub(int x) {
15        // do something
16    }
17    vector<query> Q;
18    void add_query(int l, int r, int id) {
19        // [l, r)
20        Q.push_back({l, r, id});
21        ans.push_back(0);
22    }
23    void run() {
24        sort(Q.begin(), Q.end());
25        int pl = 0, pr = 0;
26        cur_ans = 0;
27        for (query &i : Q) {
28            while (pl > i.l)
29                add(a[--pl]);
30            while (pr < i.r)
31                add(a[pr++]);
32            while (pl < i.l)
33                sub(a[pl++]);
34            while (pr > i.r)
35                sub(a[pr--]);
36            ans[i.id] = cur;
37        }
38    }
39 };

```

6.8 number split

```

1 // dp[i][x] = 要將整數 x 拆成 i 堆的「組合數」
2 dp[i + 1][x + 1] += dp[i][x]; // 創造新的一堆
3 dp[i][x + i] += dp[i][x]; // 把每一堆都增加 1

```

6.9 sos dp

```

1 // 時間複雜度為 O(n * 2 ^ n)
2 // 計算 dp[i] = i 所有 bit mask 子集的和
3 for (int i = 0; i < n; i++)
4     for (int mask = 0; mask < (1 << n); mask++)
5         if ((mask >> i) & 1)
6             dp[mask] += dp[mask ^ (1 << i)];

```

6.10 subset

```

1 for (int i = 0; i < (1 << n); i++) {
2     for (int m = i; m; m = (m - 1) & i) {
3         // m 是 i 的非空子集
4     }
5 }

```

6.11 xor basis

```

1 // 給一數字集合 S · 求能 XOR 出多少數字
2 // 答案等於 2 ^ {basis 的大小}
3 vector<int> basis;
4 void add_vector(int x) {
5     for (int v : basis) x = min(x, x ^ v);
6     if (x) basis.push_back(x);
7 }
8 // 給一數字集合 S · 求能不能 XOR 出 x
9 bool check(int x) {
10    for (int v : basis) x = min(x, x ^ v);
11    return x;
12 }
13 // 給一數字集合 S · 求 XOR 出最大的數字
14 int get_max() {
15    int ans = 0;
16    for (auto v : basis) ans = max(ans, ans ^ v);
17    return ans;
18 }

```

7 polynomial

7.1 FFT

```

1 using T = complex<double>;
2 const double PI = acos(-1);
3 struct FFT {
4     T w[N];
5     FFT() {
6         T dw = {cos(2 * PI / N), sin(2 * PI / N)};
7         w[0] = 1;
8         for (int i = 1; i < N; ++i) w[i] = w[i - 1] * dw;
9     }
10    void operator()(vector<T>& a, bool inv = false) {
11        // see NTT, replace LL with T
12        if (inv) {
13            reverse(1 + all(a));
14            T invn = 1.0 / n;
15            for (int i = 0; i < n; ++i) a[i] = a[i] * invn;
16        }
17    } ntt;
18 } // after mul, round i.real()

```

7.2 NTT

```

1 // mul, add, sub, Pow
2 struct NTT {
3     int w[N];
4     NTT() {
5         int dw = Pow(G, (mod - 1) / N);
6         w[0] = 1;
7         for (int i = 1; i < N; ++i)
8             w[i] = mul(w[i - 1], dw);
9     } // 0 <= a[i] < P
10    void operator()(vector<int>& a, bool inv = false) {
11        int n = sz(a);
12        for (int j = 1, x = 0; j < n - 1; ++j) {
13            for (int k = n >> 1; (x ^= k) < k; k >>= 1);
14            if (j < x) swap(a[x], a[j]);
15        }
16        for (int L = 2; L <= n; L <= 1) {
17            int dx = N / L, dl = L >> 1;
18            for (int i = 0; i < n; i += L) {
19                for (int j = i, x = 0; j < i + dl; ++j, x += dx) {
20                    int tmp = mul(a[j + dl], w[x]);
21                    a[j + dl] = sub(a[j], tmp);
22                    a[j] = add(a[j], tmp);
23                }
24            }
25        }
26        if (inv) {
27            reverse(1 + all(a));
28            int invn = Pow(n, mod - 2);
29            for (int i = 0; i < n; ++i)
30                a[i] = mul(a[i], invn);
31        }
32    } ntt;
33 }

```

7.3 NTT prime

Prime	Root	Prime	Root
7681	17	167772161	3
12289	11	104857601	3
40961	3	985661441	3
65537	3	998244353	3
786433	10	1107296257	10
5767169	3	2013265921	31
7340033	3	2810183681	11
23068673	3	2885681153	3
469762049	3	605028353	3
2061584302081	7	1945555039024054273	5
2748779069441	3	9223372036737335297	3

7.4 operation

```

1 typedef vector<int> Poly;
2 Poly Mul(Poly a, Poly b, int bound = N) {
3     int m = sz(a) + sz(b) - 1, n = 1;
4     while (n < m) n <= 1;
5     a.resize(n), b.resize(n);
6     ntt(a), ntt(b);
7     for (int i = 0; i < n; ++i) a[i] = mul(a[i], b[i]);
8     ntt(a, true), a.resize(min(m, bound));
9     return a;
10 }
11 Poly Inverse(Poly a) {
12     // O(NLogN), a[0] != 0
13     int n = sz(a);
14     Poly res(1, Pow(a[0], mod - 2));
15     for (int m = 1; m < n; m <= 1) {
16         if (n < m * 2) a.resize(m * 2);
17         Poly v1(a.begin(), a.begin() + m * 2), v2 = res;
18         v1.resize(m * 4), v2.resize(m * 4);
19         ntt(v1), ntt(v2);
20         for (int i = 0; i < m * 4; ++i)
21             v1[i] = mul(mul(v1[i], v2[i]), v2[i]);
22         ntt(v1, true);
23         res.resize(m * 2);
24         for (int i = 0; i < m; ++i)
25             res[i] = add(res[i], res[i]);
26         for (int i = 0; i < m * 2; ++i)
27             res[i] = sub(res[i], v1[i]);
28     }
29     res.resize(n);
30     return res;
31 }
32 pair<Poly, Poly> Divide(Poly a, Poly b) {
33     // a = bQ + R, O(NLogN), b.back() != 0
34     int n = sz(a), m = sz(b), k = n - m + 1;
35     if (n < m) return {{0}, a};
36     Poly ra = a, rb = b;
37     reverse(all(ra)), ra.resize(k);
38     reverse(all(rb)), rb.resize(k);
39     Poly Q = Mul(ra, Inverse(rb), k);
40     reverse(all(Q));
41     Poly res = Mul(b, Q), R(m - 1);
42     for (int i = 0; i < m - 1; ++i)
43         R[i] = sub(a[i], res[i]);
44     return {Q, R};
45 }
46 Poly SqrtImpl(Poly a) {
47     if (a.empty()) return {0};
48     int z = QuadraticResidue(a[0], mod), n = sz(a);
49     if (z == -1) return {-1};
50     Poly q(1, z);
51     const int inv2 = (mod + 1) / 2;
52     for (int m = 1; m < n; m <= 1) {
53         if (n < m * 2) a.resize(m * 2);
54         q.resize(m * 2);
55         Poly f2 = Mul(q, q, m * 2);
56         for (int i = 0; i < m * 2; ++i)
57             f2[i] = sub(f2[i], a[i]);
58         f2 = Mul(f2, Inverse(q), m * 2);
59         for (int i = 0; i < m * 2; ++i)
60             q[i] = sub(q[i], mul(f2[i], inv2));
61     }
62     q.resize(n);
63     return q;
64 }
65 Poly Sqrt(Poly a) {
66     // O(NLogN), return {-1} if not exists
67     int n = sz(a), m = 0;
68     while (m < n && a[m] == 0) m++;
69     if (m == n) return Poly(n);
70     if (m & 1) return {-1};
71     Poly s = SqrtImpl(Poly(a.begin() + m, a.end()));
72     if (s[0] == -1) return {-1};
73     Poly res(n);
74     for (int i = 0; i < sz(s); ++i)
75         res[i + m / 2] = s[i];
76     return res;
77 }

```

```

78 Poly Derivative(Poly a) {
79     int n = sz(a);
80     Poly res(n - 1);
81     for (int i = 0; i < n - 1; ++i)
82         res[i] = mul(a[i + 1], i + 1);
83     return res;
84 }
85 Poly Integral(Poly a) {
86     int n = sz(a);
87     Poly res(n + 1);
88     for (int i = 0; i < n; ++i)
89         res[i + 1] = mul(a[i], Pow(i + 1, mod - 2));
90     return res;
91 }
92 Poly Ln(Poly a) {
93     // O(NLogN), a[0] = 1
94     int n = sz(a);
95     if (n == 1) return {0};
96     Poly d = Derivative(a);
97     a.pop_back();
98     return Integral(Mul(d, Inverse(a), n - 1));
99 }
100 Poly Exp(Poly a) {
101     // O(NLogN), a[0] = 0
102     int n = sz(a);
103     Poly q(1, 1);
104     a[0] = add(a[0], 1);
105     for (int m = 1; m < n; m <= 1) {
106         if (n < m * 2) a.resize(m * 2);
107         Poly g(a.begin(), a.begin() + m * 2), h(all(q));
108         h.resize(m * 2), h = Ln(h);
109         for (int i = 0; i < m * 2; ++i)
110             g[i] = sub(g[i], h[i]);
111         q = Mul(g, q, m * 2);
112     }
113     q.resize(n);
114     return q;
115 }
116 Poly PolyPow(Poly a, ll k) {
117     int n = sz(a), m = 0;
118     Poly ans(n, 0);
119     while (m < n && a[m] == 0) m++;
120     if (k && m && (k >= n || k * m >= n)) return ans;
121     if (m == n) return ans[0] = 1, ans;
122     int lead = m * k;
123     Poly b(a.begin() + m, a.end());
124     int base = Pow(b[0], k), inv = Pow(b[0], mod - 2);
125     for (int i = 0; i < n - m; ++i)
126         b[i] = mul(b[i], inv);
127     b = Ln(b);
128     for (int i = 0; i < n - m; ++i)
129         b[i] = mul(b[i], k % mod);
130     b = Exp(b);
131     for (int i = lead; i < n; ++i)
132         ans[i] = mul(b[i - lead], base);
133     return ans;
134 }
135 vector<int> Evaluate(Poly a, vector<int> x) {
136     if (x.empty()) return {};
137     int n = sz(x);
138     vector<Poly> up(n * 2);
139     for (int i = 0; i < n; ++i)
140         up[i + n] = {sub(0, x[i]), 1};
141     for (int i = n - 1; i > 0; --i)
142         up[i] = Mul(up[i * 2], up[i * 2 + 1]);
143     vector<Poly> down(n * 2);
144     down[1] = Divide(a, up[1]).second;
145     for (int i = 2; i < n * 2; ++i)
146         down[i] = Divide(down[i >> 1], up[i]).second;
147     Poly y(n);
148     for (int i = 0; i < n; ++i) y[i] = down[i + n][0];
149     return y;
150 }
151 Poly Interpolate(vector<int> x, vector<int> y) {
152     int n = sz(x);
153     vector<Poly> up(n * 2);
154     for (int i = 0; i < n; ++i)
155         up[i + n] = {sub(0, x[i]), 1};
156     for (int i = n - 1; i > 0; --i)
157         up[i] = Mul(up[i * 2], up[i * 2 + 1]);
158     Poly a = Evaluate(Derivative(up[1]), x);
159     for (int i = 0; i < n; ++i)
160         a[i] = mul(y[i], Pow(a[i], mod - 2));
161     vector<Poly> down(n * 2);
162     for (int i = 0; i < n; ++i) down[i + n] = {a[i]};
163     for (int i = n - 1; i > 0; --i) {
164         Poly lhs = Mul(down[i * 2], up[i * 2 + 1]);
165         Poly rhs = Mul(down[i * 2 + 1], up[i * 2]);
166         down[i].resize(sz(lhs));
167         for (int j = 0; j < sz(lhs); ++j)
168             down[i][j] = add(lhs[j], rhs[j]);
169     }
170     return down[1];
171 }
172 Poly TaylorShift(Poly a, int c) {
173     // return sum a_i(x + c)^i;
174     // fac[i] = i!, facp[i] = inv(i!)

```

```

175     int n = sz(a);
176     for (int i = 0; i < n; ++i) a[i] = mul(a[i], fac[i]);
177     reverse(all(a));
178     Poly b(n);
179     int w = 1;
180     for (int i = 0; i < n; ++i)
181         b[i] = mul(facp[i], w), w = mul(w, c);
182     a = Mul(a, b, n), reverse(all(a));
183     for (int i = 0; i < n; ++i) a[i] = mul(a[i], facp[i]);
184     return a;
185 }
186 vector<int> SamplingShift(vector<int> a, int c, int m){
187     // given f(0), f(1), ..., f(n - 1)
188     // return f(c), f(c + 1), ..., f(c + m - 1)
189     int n = sz(a); // 4d649d
190     for (int i = 0; i < n; ++i) a[i] = mul(a[i], facp[i]);
191     Poly b(n);
192     for (int i = 0; i < n; ++i) {
193         b[i] = facp[i];
194         if (i & 1) b[i] = sub(0, b[i]);
195     }
196     a = Mul(a, b, n);
197     for (int i = 0; i < n; ++i) a[i] = mul(a[i], fac[i]);
198     reverse(all(a));
199     int w = 1;
200     for (int i = 0; i < n; ++i)
201         b[i] = mul(facp[i], w), w = mul(w, sub(c, i));
202     a = Mul(a, b, n);
203     reverse(all(a));
204     for (int i = 0; i < n; ++i) a[i] = mul(a[i], facp[i]);
205     a.resize(m), b.resize(m);
206     for (int i = 0; i < m; ++i) b[i] = facp[i];
207     a = Mul(a, b, m);
208     for (int i = 0; i < m; ++i) a[i] = mul(a[i], fac[i]);
209     return a;
210 }
211 Poly power_proj(vector<int> w, Poly f, int m) {
212     // return sum_j w_j[x^j]f^i for i=0,1,...,m
213     int n = 1;
214     while (n < sz(f)) n <= 1;
215     f.resize(n), w.resize(n), reverse(all(w));
216     int k = 1, n2 = 2 * n, n4 = 4 * n;
217     Poly _P(n2), _Q(n2);
218     for (int i = 0; i < n; ++i)
219         _P[i] = w[i], _Q[i] = sub(0, f[i]);
220     while (n > 1) {
221         Poly R(n2);
222         for (int i = 0; i < n2; ++i)
223             R[i] = (i & 1 ? sub(0, _Q[i]) : _Q[i]);
224         Poly PQ = Mul(_P, R), QQ = Mul(_Q, R);
225         PQ.resize(n4), QQ.resize(n4);
226         for (int i = 0, j = n2; i < n2; ++i, ++j) {
227             PQ[j] = add(PQ[j], _P[i]);
228             QQ[j] = add(QQ[j], add(_Q[i], R[i]));
229         }
230         fill(all(_P), 0), fill(all(_Q), 0);
231         for (int j = 0; j < 2 * k; ++j)
232             for (int i = 0; i < n / 2; ++i) {
233                 _P[n * j + i] = PQ[2 * n * j + 2 * i + 1];
234                 _Q[n * j + i] = QQ[2 * n * j + 2 * i + 0];
235             }
236         n /= 2, k *= 2;
237     }
238     Poly p(k);
239     for (int i = 0; i < k; ++i) p[i] = _P[2 * i];
240     reverse(all(p)), p.resize(m + 1);
241     return p;
242 }

```

8 string

8.1 hash

```

1 const int p = 1e6 + 3;
2 const int mod = 1e9 + 7;
3 int hash(string &s) {
4     int res = 0;
5     for (int i = 0; i < s.size(); i++) {
6         res *= p;
7         res += s[i] - 'a' + 1;
8         res %= mod;
9     }
10    return res;
11 }
12 int pow_p[1000005];
13 vector<int> rh;
14 void build(string &s) {
15     rh.resize(s.size() + 1);
16     pow_p[0] = 1;
17     for (int i = 1; i <= s.size(); i++)
18         pow_p[i] = (pow_p[i - 1] * p) % mod;
19     rh[0] = s[0] - 'a' + 1;

```

```

20 for (int i = 1; i < s.size(); i++) {
21     rh[i] = rh[i - 1] * p + (s[i] - 'a' + 1);
22     rh[i] %= mod;
23 }
24 }
25 int query(int l, int r) {
26     int res = rh[r] - (l > 0 ? rh[l - 1] * pow_p[r - l + 1] : 0);
27     res = (res % mod + mod) % mod;
28     return res;
29 }

```

8.2 kmp

```

1 vector<int> F;
2 void build_failure_function(string &s) {
3     F.clear();
4     F.resize(s.size(), -1);
5     for (int i = 1; i < s.size(); i++) {
6         int j = F[i - 1];
7         while (j != -1 && s[i] != s[j + 1])
8             j = F[j];
9         if (s[i] == s[j + 1])
10            F[i] = j + 1;
11     }
12 }
13 void KMP_matching(string &a, string &b) {
14     for (int i = 0, j = -1; i < a.size(); i++) {
15         while (j != -1 && a[i] != b[j + 1])
16             j = F[j];
17         if (a[i] == b[j + 1])
18             j++;
19         if (j + 1 == b.size()) {
20             cout << "found a matching start at " << i - j << "\n";
21             j = F[j];
22         }
23     }
24 }

```

8.3 manacher

```

1 string s;
2 int extend(int l, int r, int N) {
3     int i = 0;
4     while (l - i >= 0 && r + i < N && s[l - i] == s[r + i])
5         i++;
6     return i;
7 }
8 int Longest_Palindromic_Substring(string &t) {
9     int N = t.length();
10    s.resize(2 * N + 1, '$');
11    for (int i = 0; i < N; i++)
12        s[2 * i + 1] = t[i];
13    N = 2 * N + 1;
14    vector<int> res;
15    res.resize(N, 0);
16    res[0] = 1;
17    for (int i = 1, mid = 0, R = 0; i < N; i++) {
18        int j = mid - (i - mid);
19        int lst = R + 1 - i;
20        if (i > R) {
21            res[i] = extend(i, i, N);
22            mid = i;
23            R = i + res[i] - 1;
24        } else if (res[j] == lst) {
25            res[i] = lst + extend(i - R, i + R, N);
26            mid = i;
27            R = i + res[i] - 1;
28        } else {
29            res[i] = min(res[j], lst);
30        }
31    }
32    int mx = -1, idx = -1;
33    for (int i = 0; i < N; i++) {
34        if (res[i] > mx) {
35            mx = res[i];
36            idx = i;
37        }
38    }
39    cout << "Longest Palindromic Substring Length is " << (mx + 1) / 2 << "\n";
40    for (int i = idx - res[idx] + 1; i <= idx + res[idx] + 1; i++)
41        if (i & 1)
42            cout << s[i];
43    cout << "\n";
44 }

```

8.4 string trie

```

1 struct node {
2     node* link[26] = {nullptr};
3     bool flag = false;
4     bool contain_key(char ch) { return (link[ch - 'a'] != nullptr); }
5     void put(char ch, node* _node) { link[ch - 'a'] = _node; }
6     node* get(char ch) { return link[ch - 'a']; }
7     void set_end() { flag = true; }
8     bool is_end() { return flag; }
9 };
10 struct Trie {
11     node* root;
12     Trie() { root = new node(); }
13     void insert(string s) { // insert a string
14         node* now = root;
15         for (int i = 0; i < (int)s.size(); i++) {
16             if (!now->contain_key(s[i])) now->put(s[i], new node());
17             now = now->get(s[i]);
18         }
19         now->set_end();
20     }
21     // return if there is a word of the given string
22     bool query(string s) {
23         node* now = root;
24         for (int i = 0; i < (int)s.size(); i++) {
25             if (!now->contain_key(s[i])) return false;
26             now = now->get(s[i]);
27         }
28         return now->is_end();
29     }
30     // return if there is any word that start with the given
31     // prefix
32     bool query_prefix(string s) {
33         node* now = root;
34         for (int i = 0; i < (int)s.size(); i++) {
35             if (!now->contain_key(s[i])) return false;
36             now = now->get(s[i]);
37         }
38         return true;
39     }
40 };

```

8.5 xor trie

```

1 struct node {
2     node* link[2];
3     bool contain_key(int val) { return (link[val] != nullptr); }
4     node* get(int val) { return link[val]; }
5     void put(int val, node* _node) { link[val] = _node; }
6 };
7 struct Trie {
8     node* root;
9     Trie() { root = new node(); }
10    void update(int x) {
11        node* now = root;
12        for (int i = 30; i >= 0; i--) {
13            if (x & (1 << i)) {
14                if (!now->contain_key(1)) now->put(1, new node());
15                now = now->get(1);
16            } else {
17                if (!now->contain_key(0)) now->put(0, new node());
18                now = now->get(0);
19            }
20        }
21    }
22    int query(int x) { // query max xor with number x
23        node* now = root;
24        int mx = 0;
25        for (int i = 30; i >= 0; i--) {
26            int nowbit = (x >> i) & 1;
27            int target = nowbit ^ 1;
28            if (now->contain_key(target)) {
29                mx += (1 << i);
30                now = now->get(target);
31            } else {
32                now = now->get(nowbit);
33            }
34        }
35        return mx;
36    }
37 };

```

8.6 z

```

1 vector<int> Z;
2 void calculate_Z(string &s) {
3     Z.clear();
4     Z.resize(s.size(), 0);

```

```

5 | int l = 0, r = 0;
6 | for (int i = 1; i < s.size(); i++) {
7 |     if (i <= r)
8 |         Z[i] = min(Z[i - 1], r - i + 1);
9 |     while (i + Z[i] < s.size() && s[i + Z[i]] == s[Z[i]])
10 |         Z[i]++;
11 |     if (i + Z[i] - 1 > r) {
12 |         l = i; r = i + Z[i] - 1;
13 |     }
14 | }
15 | }
16 | void Z_matching(string &a, string &b) {
17 |     string res = "";
18 |     int i = 0;
19 |     for (char ch : b)
20 |         res[i++] = ch;
21 |     res[i++] = '_';
22 |     for (char ch : a)
23 |         res[i++] = ch;
24 |     calculate_Z(res);
25 |     for (int i = 0; i < Z.size(); i++) {
26 |         if (Z[i] == b.size())
27 |             cout << "found a matching start at " << i - b.size() - 1
28 |                 << "\n";
29 |     }
}

```

9 tree

9.1 hld

```

1 | const int N = 100005;
2 | vector<int> G[N];
3 | struct HLD {
4 |     vector<int> pa, sz, depth, mxson, topf, id;
5 |     int n, idcnt = 0;
6 |     HLD(int _n) : n(_n), pa(_n + 1), sz(_n + 1), depth(_n + 1),
7 |                 mxson(_n + 1), topf(_n + 1), id(_n + 1) {}
8 |     void dfs1(int v = 1, int p = -1) {
9 |         pa[v] = p; sz[v] = 1; mxson[v] = 0;
10 |        depth[v] = (p == -1 ? 0 : depth[p] + 1);
11 |        for (int u : G[v]) {
12 |            if (u == p) continue;
13 |            dfs1(u, v);
14 |            sz[v] += sz[u];
15 |            if (sz[u] > sz[mxson[v]]) mxson[v] = u;
16 |        }
17 |    }
18 |    void dfs2(int v = 1, int top = 1) {
19 |        id[v] = ++idcnt;
20 |        topf[v] = top;
21 |        if (mxson[v]) dfs2(mxson[v], top);
22 |        for (int u : G[v]) {
23 |            if (u == mxson[v] || u == pa[v]) continue;
24 |            dfs2(u, u);
25 |        }
26 |    }
27 |    int path_query(int a, int b) {
28 |        int res = 0;
29 |        while (topf[a] != topf[b]) {
30 |            if (depth[topf[a]] < depth[topf[b]]) swap(a, b);
31 |            res = max(res, query(id[topf[a]], id[a])); // query : l =
32 |                id[topf[a]], r = id[a]
33 |            a = pa[topf[a]];
34 |        }
35 |        if (depth[a] < depth[b]) swap(a, b);
36 |        res = max(res, query(id[b], id[a])); // query : l = id[b],
37 |            r = id[a]
38 |        return res;
39 |    }
}

```